JUNIOR BACKEND DEVELOPMENT ASSIGNMENT

Personal Expense & Budget Management API

Independent practical exercise | Junior level | Suggested duration: 4–6 working days

Objective. Design and build a complete secured REST API that lets users manage personal expenses, organize them into categories, and monitor monthly budgets. Make technical decisions independently; no predefined entity model or implementation solution is supplied.

1. Business scenario

Users register accounts, record daily expenses (e.g., groceries, transportation, restaurants), and optionally set monthly budgets for expense categories. The API allows users to review their spending and remaining budgets.

Date	Category	Description	Amount
Oct 2	Groceries	Supermarket	$75
Oct 4	Transportation	Fuel	$40
Oct 7	Groceries	Weekly shopping	$55
Oct 10	Restaurants	Dinner	$30

Example: If the October grocery budget is $200, and grocery spending totals $130, the remaining budget is $70.

2. Functional requirements

A. User management

- Users can register and log in using email and password.
- Each user maintains their own data; no user may access another user’s private records.

B. Expense categories

- Users can create, list, update, and delete categories, each with a name and optional description.
- Category names must be unique per user, but different users may use identical names.
- Deleting a category must not cause accidental loss of existing expenses.

C. Expense management

- Create, list, update, and delete expenses with amount, description, date, and category.
- Each expense must reference a category owned by the authenticated user.
- Support filtering expenses by category and date range.

D. Monthly budgets

- Users can define and update a budget for a category, month, and year.

- Only one budget is allowed per user/category/month/year; budgets must be positive.
- Expenses remain valid even when no budget exists.

E. Monthly summary

- Return total spending per selected month, and per-category spending, configured budget, and remaining amount (where a budget exists).

3. Database design assignment

Before writing entities or repository code, submit an ERD showing tables, columns, keys, relationships, and cardinalities. Identify appropriate constraints, uniqueness rules, and referential behavior.

- Ownership: categories, expenses, and budgets are scoped to an individual user.
- Uniqueness: category names are unique within a user account, not globally.
- Calendar periods: monthly budgets must be linked to a specific month and year.
- Integrity: prevent invalid cross-user references and unwanted cascade deletion.
- Money: choose a database representation that avoids floating-point inaccuracies.

4. Mandatory technologies

Technology	Required use
Java 21 / Spring Boot	REST API implementation
Spring Data JPA / Hibernate	Persistence and entity mapping
PostgreSQL	Relational database
Flyway	Version-controlled database migrations
Spring Security + JWT	Authentication and authorization
Git	Version control and commit history
Docker + Docker Compose	Containerize the app and database

5. REST API requirements

Area	Required operations
Authentication	Register, log in
Categories	Create, list, update, delete
Expenses	Create, list, update, delete; filter by category/date
Budgets	Set, update, list
Summary	Monthly spending summary

- Use suitable HTTP methods and response status codes, DTOs, request validation, and consistent error responses.

- Require JWT authentication for protected operations and ensure access is restricted to the owner of each resource.
- Do not allow a user to create an expense referencing another user’s category, even if the ID is known.

6. JWT security requirements

- Provide registration and login endpoints; login returns a signed access token.
- Validate JWT signature and expiration on protected requests.
- Hash passwords securely; never store plaintext passwords.
- Keep signing keys and sensitive settings outside source control.
- Ensure authorization includes ownership checks, not merely valid authentication.

7. Flyway requirements

- Create an initial schema migration and at least one later migration that makes a meaningful schema change.
- Execute migrations during application startup on a fresh PostgreSQL database.
- Do not rely on Hibernate to generate or alter tables; do not edit already-applied migrations.

8. Docker and Docker Compose

- Supply a Dockerfile for the Spring Boot application and a Compose configuration for app and PostgreSQL services.
- Use environment variables for database connection and security configuration; persist PostgreSQL data in a named volume.
- Ensure the application uses the Compose network host name and starts reliably after database readiness.

Expected startup command: docker compose up --build

9. Git and documentation

- Use Git incrementally with meaningful commits, a suitable .gitignore, and no committed credentials.
- Provide README setup instructions, environment-variable examples, and API documentation or a Postman collection.

10. Expected deliverables

- ERD diagram and design assumptions
- Working Spring Boot REST API
- Flyway migrations and PostgreSQL schema
- JWT-secured endpoints with ownership checks
- Dockerfile and Docker Compose setup
- Git repository and README
- Postman collection or equivalent API documentation
- Basic automated tests for important business and security rules

11. Assessment scenarios

Scenario	Expected behavior
Register and log in	JWT returned on successful authentication
Create an expense	Expense saved for authenticated user
Duplicate category name for same user	Rejected
Use another user’s category ID	Rejected
View or change another user’s expense	Access denied
Duplicate category budget in one month	Rejected
Generate monthly summary	Totals and remaining budgets correctly computed
Present expired JWT	Protected request rejected
Start with an empty database	Flyway initializes schema
Run Docker Compose	App and database operate successfully

12. Evaluation criteria

Area	Weight
Database design and constraints	25%
REST API architecture and business logic	25%
JWT security and ownership authorization	20%
Flyway and Docker	15%
Git and documentation	10%
Automated testing and demonstration	5%

13. Suggested schedule

Day	Focus
1	Requirements, ERD, Git setup
2	Spring Boot, PostgreSQL, Flyway, entities
3	Persistence, business logic, CRUD endpoints
4	JWT, ownership checks, input validation
5	Monthly summary, Docker, Compose
6	Tests, README, final demonstration

14. Independent-work instructions

Developers may consult official documentation and learning materials, but they must be able to explain their implementation. The assessment may include a small live change, such as introducing an optional field via a new Flyway migration, and a demonstration that one authenticated user cannot access another user’s expenses.

Important: This brief specifies required behavior and assessment expectations; it intentionally does not prescribe table designs, Java classes, endpoint paths, or a JWT implementation.